Git

What is Git ?

→ Git & Github are different. It's a distributed VCS (version control system). or SCM (Source Control/Code Manager).

Eg. of VCS → Git, mercurial etc.

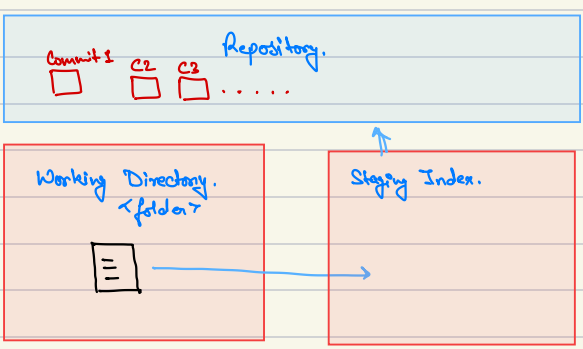
Types of VCS :

- Centralised.
- Distributed.

Advantages :

- Version Control.
- Bug fixing
- Doing non-linear developement.
- Collaborative developement.

How Git works ?



Cloning → $ git clone <url from github>

making git repo → $ git init

Status → $ git status

Making Changes :

Adding file to staging index → $ git add <filename>

Commit → $ git commit -m "My message here"

When to commit ?

→ No fixed answer. Commit after adding a functionality, or an independent feature.

Commit message :

- Short
- Explain what.
- rule of thumb → No "and".
- This commit will .... <Message>

  ✗ remove this statement. ✓
- Don't explain why.

Adding many files → $ git add .

Ignore file → Make a folder ".gitignore"

Seeing Commits :

Seeing Commits → $ git log

Just to see summary of commits → $ git log --oneline

Other variations :—

Changes in total lines → $ git log --stat

< Q > to escape.

Patch :

$ git log -p

It shows the code.

Better

$ git log --oneline

$ git show <sha>

Working on others repository]

Seeing commit history of downloaded repo. $ git log

$ git log --oneline.

Recent commit $ git show <sha>

< Q > → to exit.

< diff > command → Tells the code added / deleted before commit. [add phase]

$ git diff

Creating Versions of a software :

tag → x.y.z

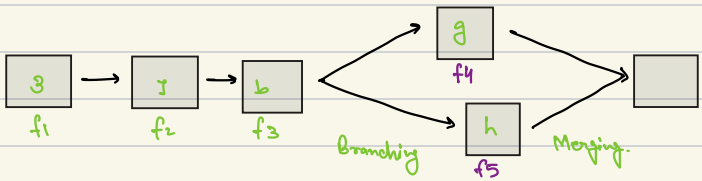
x → Major version, Backward incompatible changes.
y → Minor version, adding functionality while maintaining backward compatibility
z → patch version, small bug fixes while maintaining backward compatibility.

Annoted tag (stores info of date & time) → $ git tag -a v1.0.0
↓
vim editor
↓
fn insert
↓
message.
↓
esc
↓
:wq
↑ save ↑ quit.

Deleting tag → $ git tag -d v1.0.0
↑
Tag name.

Sticking tag to other commit → $ git tag -a v1.0.0 <sha>
↓
first insert
↓
message
↓
esc
↓
:wq

Working in non-linear flow :



When do we use branching ?
- → Scenario (Individual)
- → Scenario (Teams)

- Branch is already present (HEAD → MASTER)
- Concept of HEAD pointer.

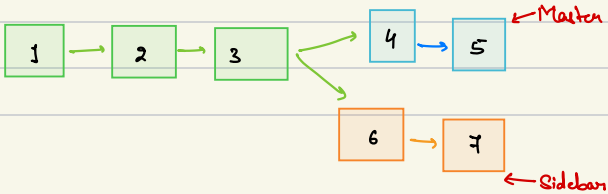
HEAD → HEAD is a reference to the most recent commit in the current branch
This means HEAD is just like a pointer that keeps track of the latest commit in your current branch.

To make branch → $ git branch <branch_name>
To show all branches → $ git branch

To cut branch in past commit → $ git branch <branch_name> <sha>
→ $ git branch error_fix 1790 ac9

Switch b/w branches → $ git checkout error_fix

→ Understanding what will come under a branch (git log)



for master → 1 2 3 4 5
for sidebar → 1 2 3 6 7 } Commit History.

$ git log --oneline

See all branches commit at once → $ git log --online --all

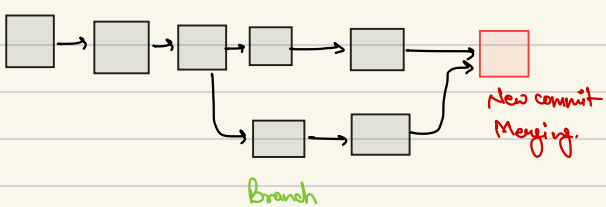
$ git log --online --all --graph

Deleting branches → $ git branch -d <branch_name>

$ git branch -D <branch_name> [w/o merge].

Merging :

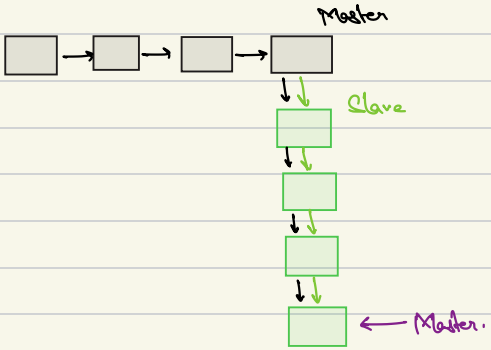
During merging a new commit takes place.



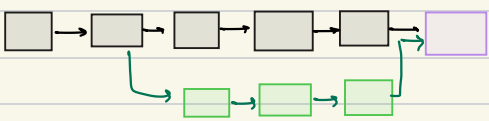
Note Merging will happen at the checked out branch.
No new branch is created.

Types of merging :
- fast-fwd merging.
- Regular.

Fast fwd :



Regular :



Merging → $ git merge <name of branch>

↓ (vim)

Esc

↓

: wq

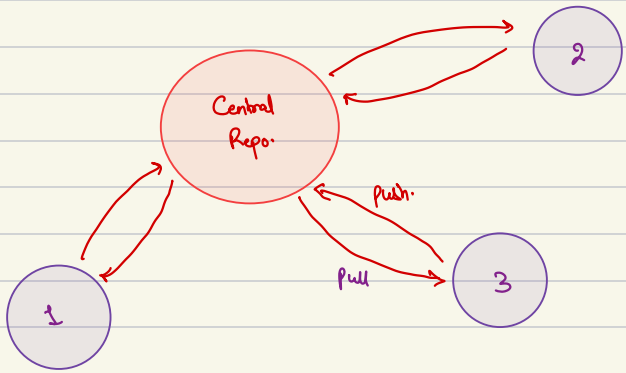
Merge Conflict :

(<< << << HEAD) everything below this line (until the next indicator) is code from current branch.

(= = == ==) is the end of the original lines, everything that follows (until the next indicator) is what's on the branch that's being merged in.

(>> >> >> heading-update) is the ending indicator of what's on the branch that's being merged in

Working with remote repo :



Github → Remote repository.

Remote push → $ git remote add origin <url>

Address of remote → $ git remote -v

Push → $ git push origin master

Pull → $ git pull origin master.